5 STRINGS

5.1 Strings

- Sequence of characters terminated by a null character '\0'.
  Syntax:
  char str_name[size];
- Initializing a string in c
  - char str[] = "Pankaj";
    Here, str is internally a pointer (holds the address of first character)
  - char str[20] = "Pankaj";
    Here, we predefine the size as 20 but we should always take one extra space along with size of string.
    i.e atleast 7 size must be there to store "Pankaj" (6+1)
  - char str[7] = {'P', 'a', 'n', 'k', 'a', 'j', '\0'};
    must set the end character as '\0'
  - chat str[] = {'p', 'a', 'n', 'k', 'a', 'j', '\0'};

5.1.1 Reading a string

- **scanf():**
  when we use scanf() to read, we use %s as a format specifier without using "&" to access the variable address because an array name acts as a pointer.

```
#include <stdio.h>
    int main (){
        char name [20];
        printf("enter your name \n");
        scanf("%s",name);
        printf("Hello %s",name);
          }
```

Output:

```
Enter your name
Pankaj Sharma
Hello Pankaj      // why not Hello Pankaj Sharma
```

The above code did not print Hello Pankaj Sharma as expected because scanf halts reading as soon as a whitespace or a newline character is encountered, and that is why it reads only Pankaj.

- In order to read a string containing space, we use gets() function. gets() ignores the whitespace & stops reading when a newline is found.

```
# include<stdio.h>
```

```
void main(){
        char name [20];
        printf("Enter your name :");
        gets(name);
        printf("Hello %s", name);
        }
Output :
        Enter your name: Pankaj Sharma
        Hello Pankaj Sharma
```

- A string literal always represent base address of a string. i.e address of first character.
- "Pankaj" represents address of character 'P'

Hence "Pankaj" [0] means 'P'

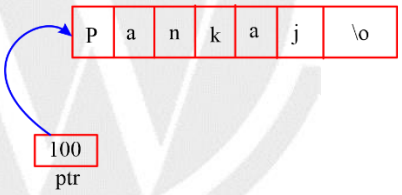
"Pankaj" [1] ≡ *("Pankaj"+1) ≡ *(Address of 'P'+1)
≡ *(Address of 'a')
≡ 'a'

5.2 Strings and pointers

- Array name represents the address of its first element. Similar to character arrays, we can create a character pointer to represent a string that will hold the starting address i.e address of first character of string.

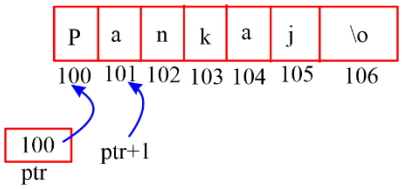
Example - 1

```
# include<stdio.h>
void main(){
        char *ptr = "pankaj"
        printf("%s",ptr);
        }
Output : pankaj
```



Example – 2

```
# include<stdio.h>
void main(){
        char *ptr = "Pankaj"
        printf("%s",ptr+1);
        }
Output: ankaj
```



5.3 Predefined functions for string

(1) strlen (): returns the length of the string passed as an argument.

(2) strcpy (): copies the contents of one string to another. strcpy (S_1,S_2) copies the content of string S_2 to string S_1.

(3) strcmp (): compares the first string with the second string. If both are same it returns 0.
strcmp (str1,str2) returns 0 if both strings are same.

(4) strcat (S_1,S_2): concat S_1 string with S_2 string and the result i.e concatenation of S_1,S_2 is stored in S_1.

5.4 Important concepts

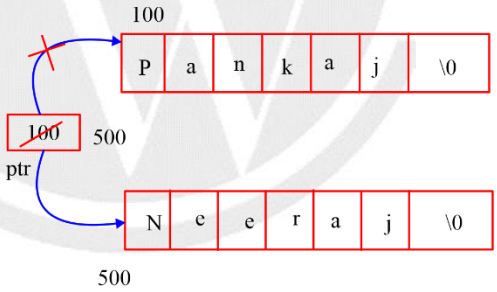
(1)
```
# include<stdio.h>
void main(){
        char name [20] = "Pankaj" ;
        name = "Neeraj" ;              // Error
        printf("%s", name);
                }
```

- Array name being a constant can't be presented at the left side of the assignment statement i.e it can't be L-value. You can't re-assign another string to array.

(2)
```
# include<stdio.h>
void main(){
                char name [20] = "Pankaj" ;
                name [1] = 'u' ;
                printf("%s", name);
                }
```
Output: Punkaj

- We are allowed to change content of array and hence we can update any character.

(3)
```
# include<stdio.h>
void main(){
                char * ptr = "Pankaj" ;
                ptr = "Neeraj" ;
                printf("%s", ptr);
                }
```
Output: Neeraj

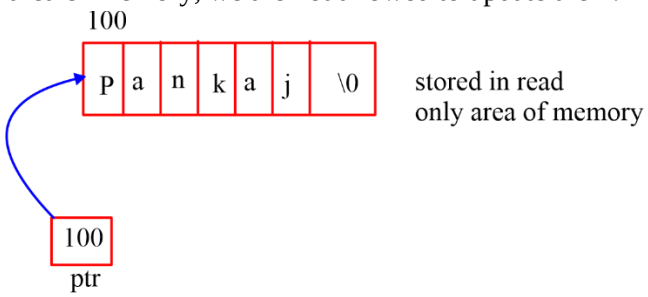


- Pointer ptr being a variable can hold different address at different time. Hence, we can assign another string to a character pointer any time.

(4)
```
# include<stdio.h>
void main()
        {
        char * ptr = "pankaj" ;
        ptr [1] = 'u' ;  // Invalid
        printf("%s", ptr);
        }
```

- String literals are stored in read only memory area i.e "pankaj" is stored first & then the address of character 'p' is given to ptr.

- As they are stored in read only area of memory, we are not allowed to update them.



❐❐❐